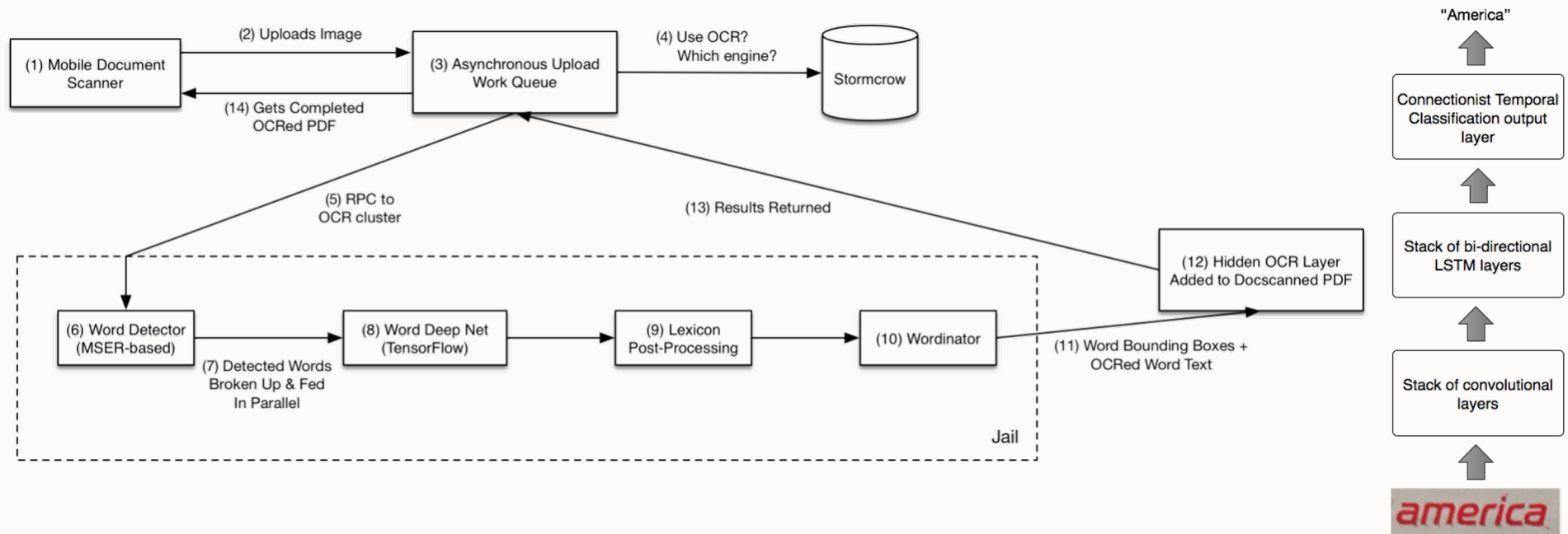




Études de cas MLOps for IoT Edges

Durée 9h

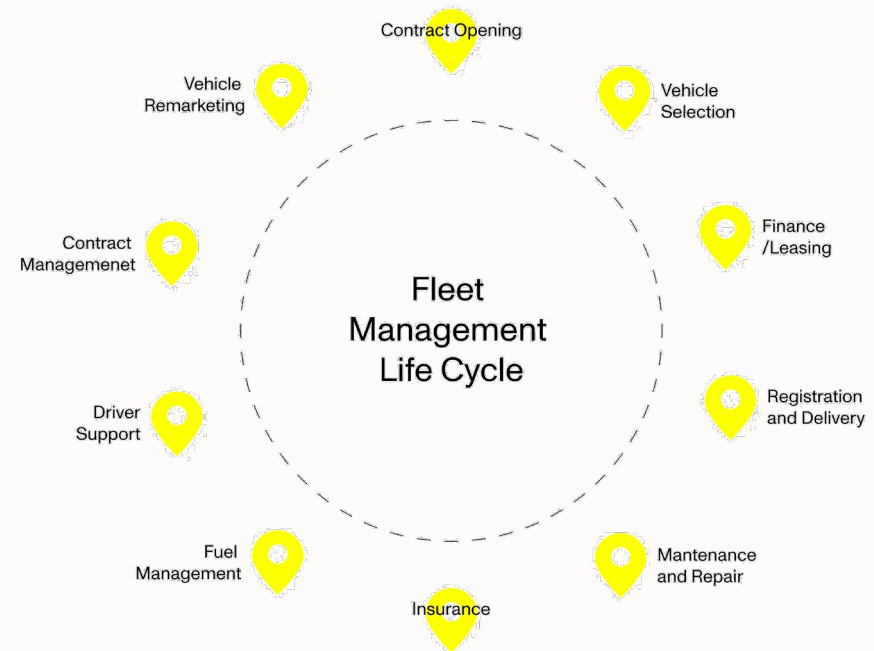
OCR Pipeline – Mobile Edge



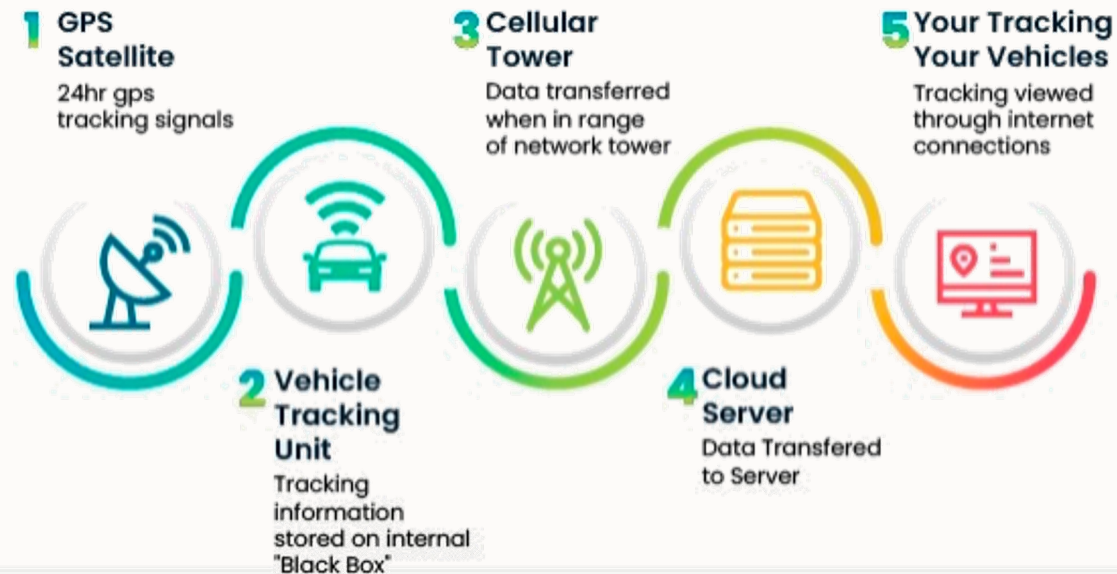
Dropbox OCR

Gestion des flottes – Fleet Management (1)

1. Contrôle temps-réel des véhicules
2. Maintenance prédictive
3. Recouvrement de véhicules égarés/volés
4. Sûreté, Sécurité et Efficacité des conduites

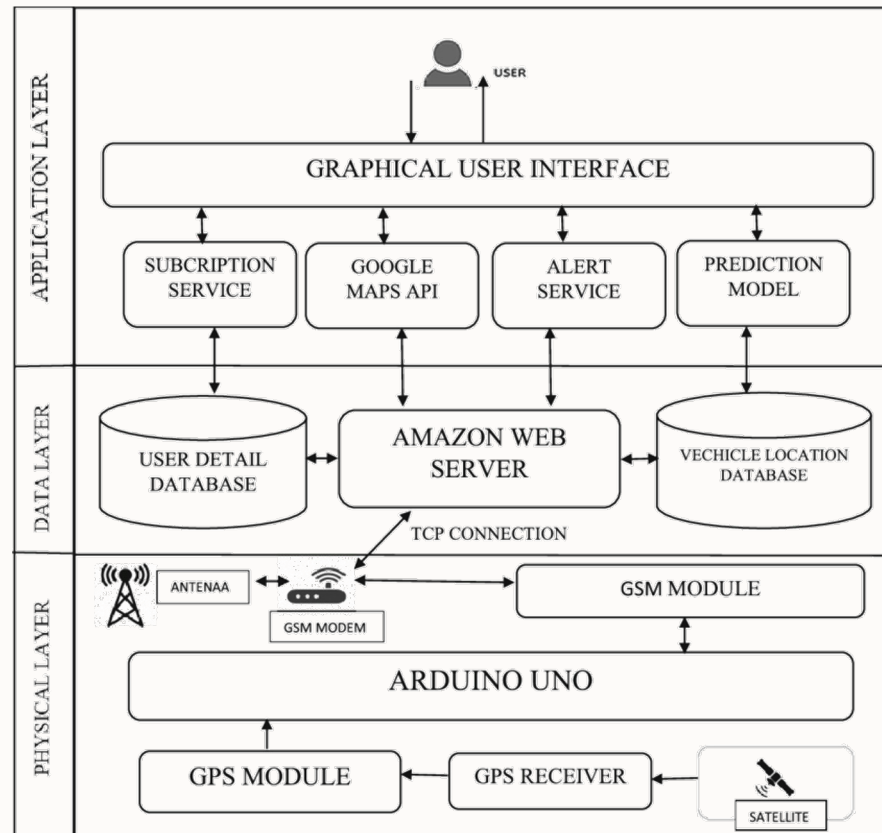


Gestion des flottes – Fleet Management (2)



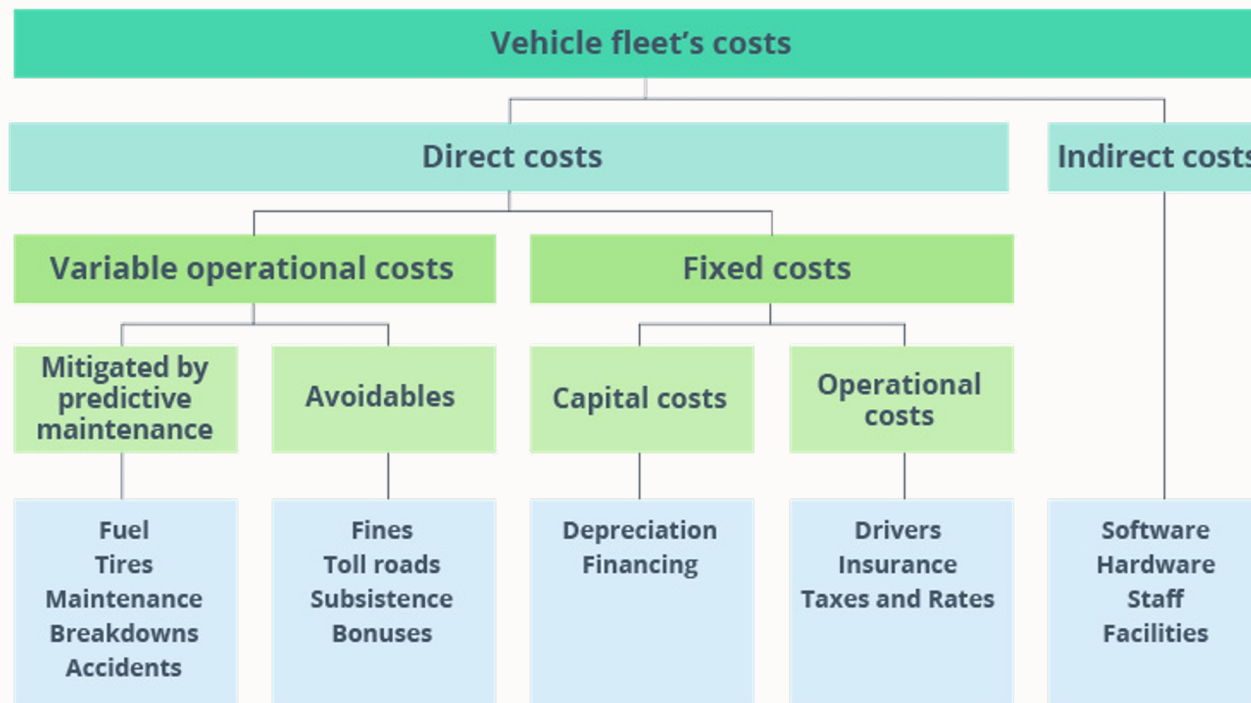
Architecture Typique

Gestion des flottes – Fleet Management (3)



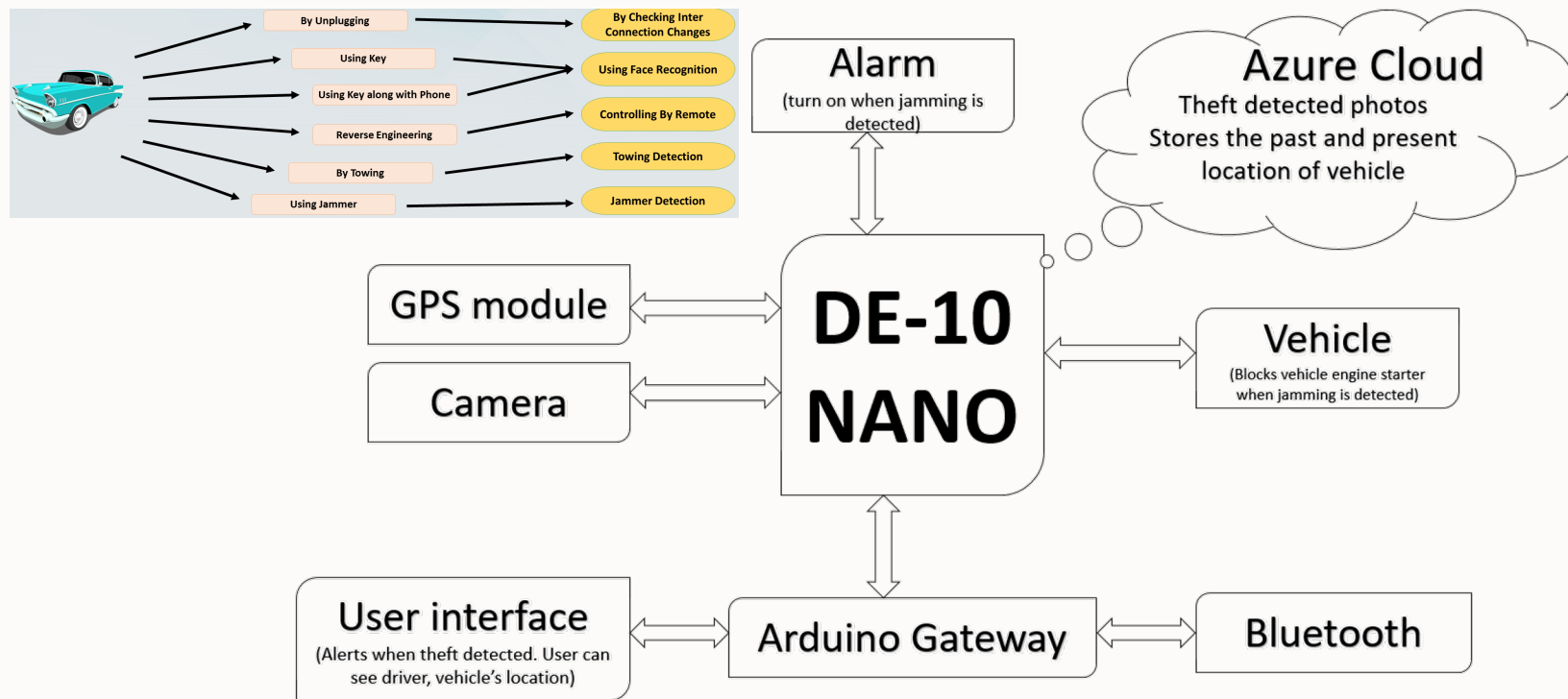
Contrôle Temps-réel des Trajectoires

Gestion des flottes – Fleet Management (4)



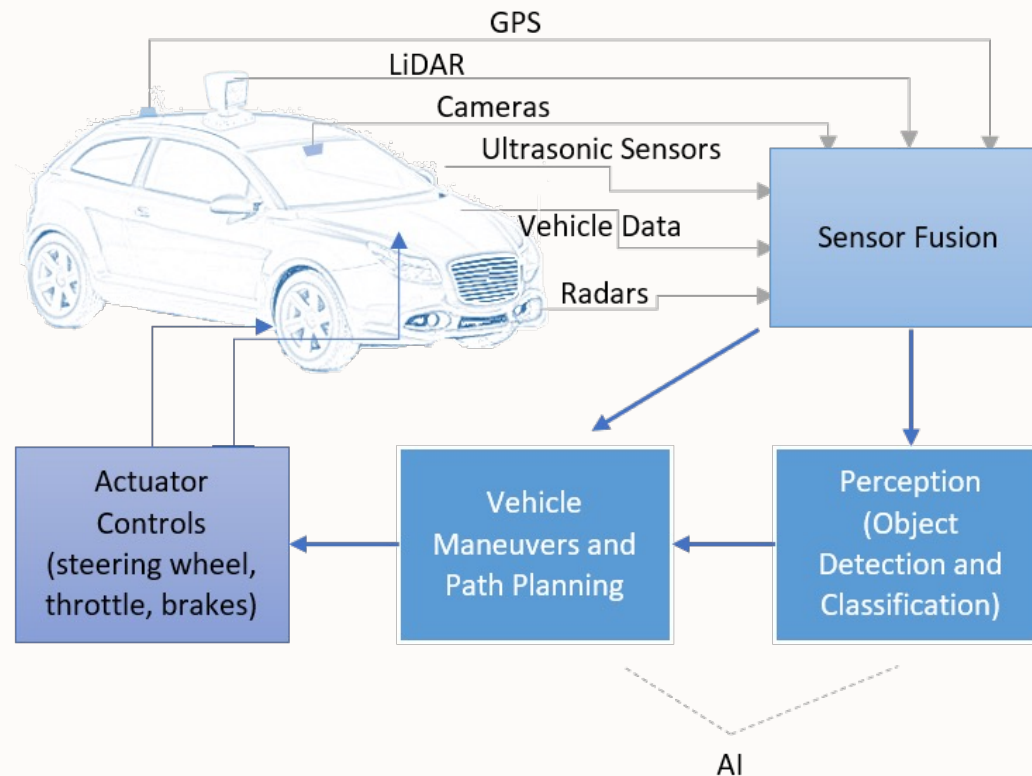
Maintenance Prédictive

Gestion des flottes – Fleet Management (5)



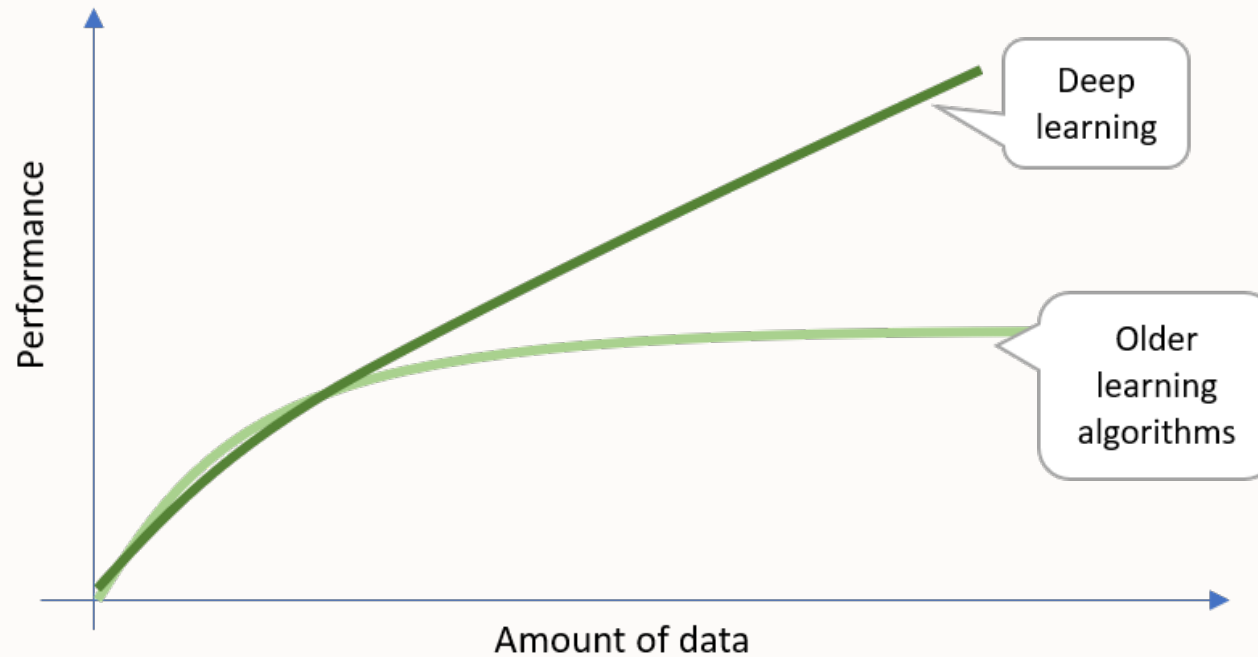
Recouvrement de véhicules

Gestion des flottes – Fleet Management (6)



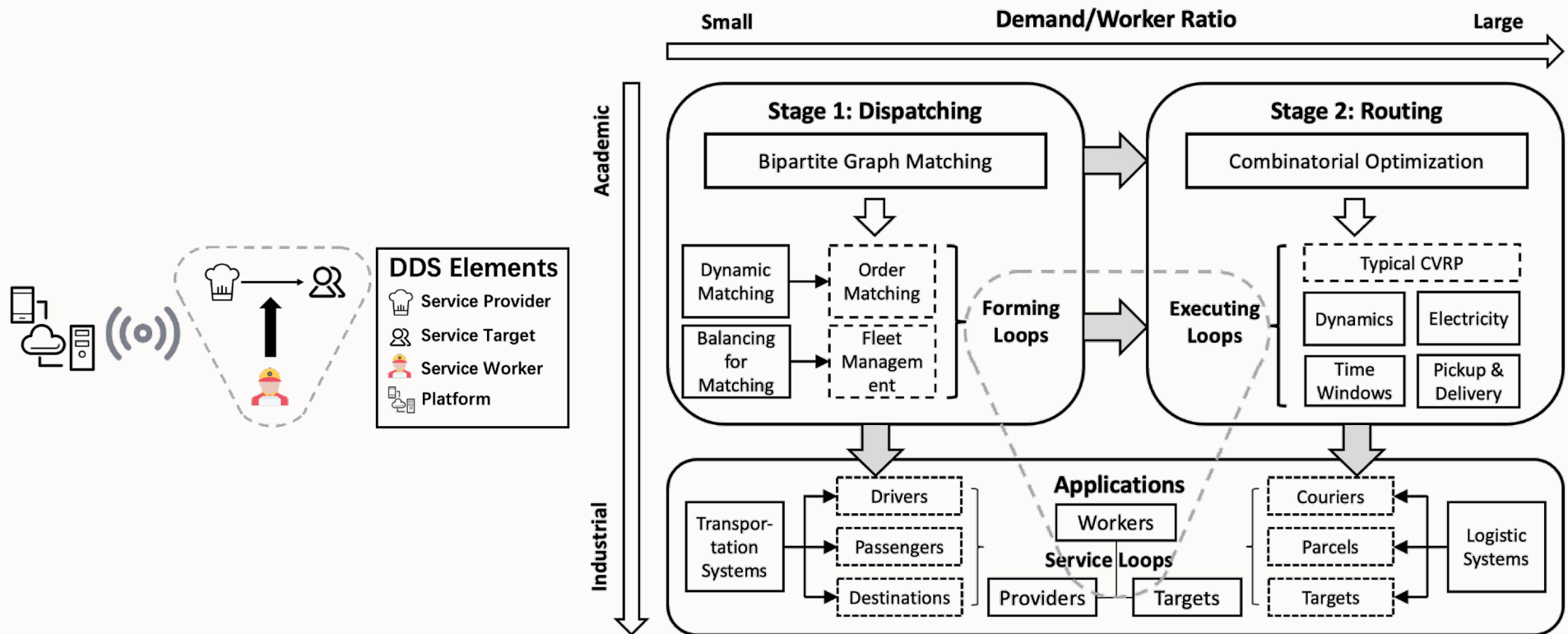
Sûreté, Sécurité et Efficacité des conduites

Gestion des flottes – Fleet Management (7)



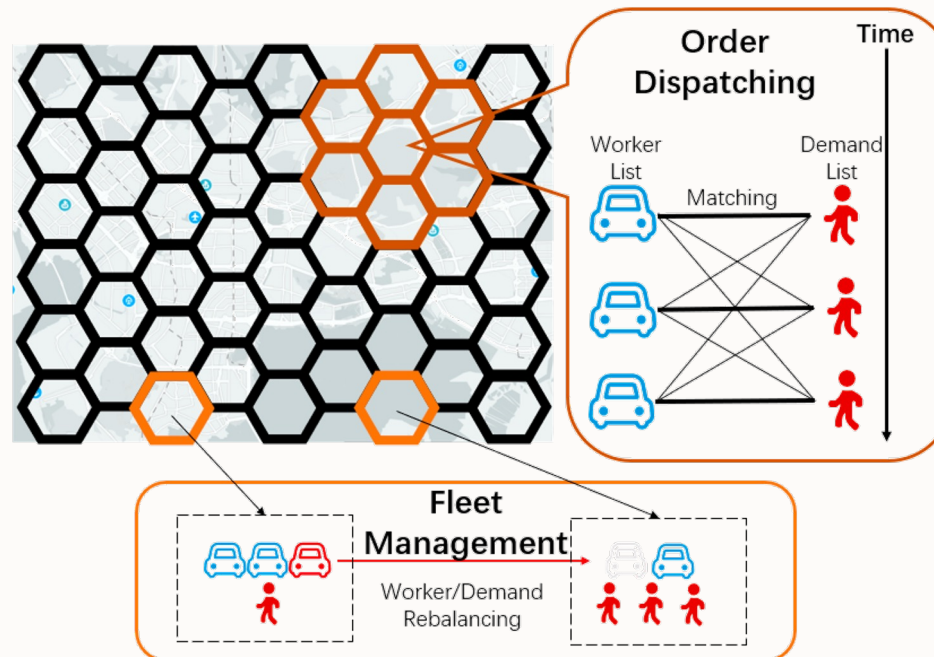
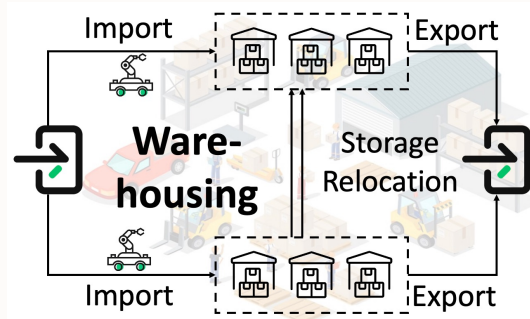
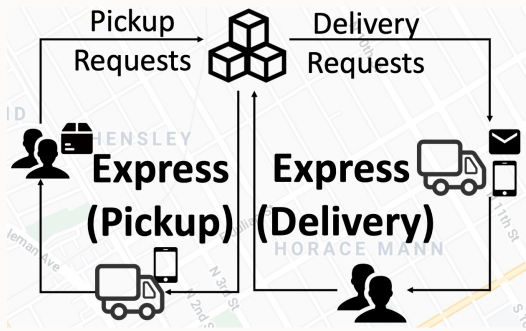
Problématique du volume des données

Gestion des flottes – Fleet Management (8)



Services Guidés par la Demande
(Demand-Driven Services)

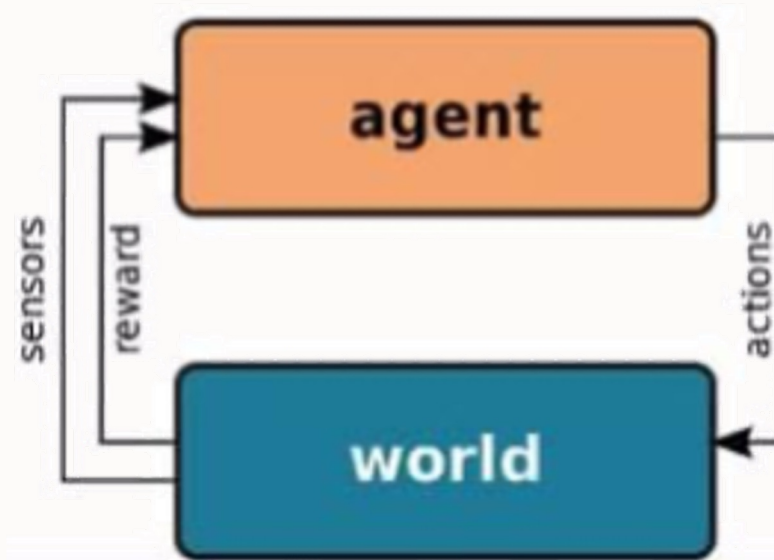
Gestion des flottes – Fleet Management (9)



Apprentissage par Renforcement (1)

- ❖ Apprendre des actions/comportements en optimisant une récompense quantitative.
- ❖ Environnement/Système représenté sous forme d'états.
- ❖ Plusieurs actions possibles à partir d'un état bien déterminé.
- ❖ Chaque action est rétribué par une récompense (positive ou négative).
- ❖ But: maximiser la somme des récompenses au cours du temps.
- ❖ Modèle/Comportement décisionnel: stratégie/politique décisionnel en fonction de l'état courant et de l'action à exécuter.

Apprentissage par Renforcement (2)



The information gathered from sensors informs/hints the agent about the state of the world

Apprentissage par Renforcement (3)

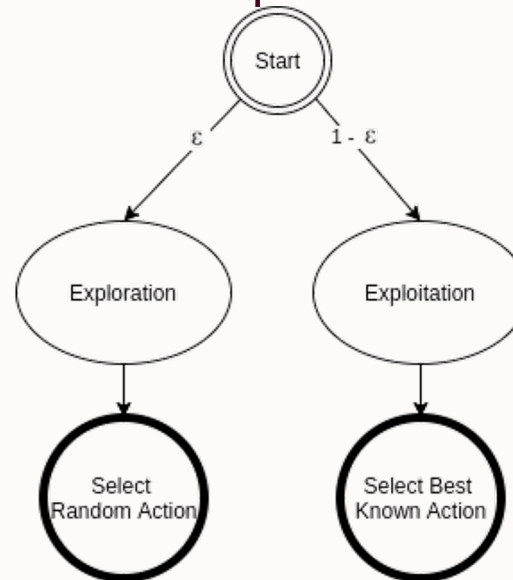
❖ Q-learning:

- ❖ Dérivé du modèle « Bush-Mosteller update rule » (1950)
- ❖ L'Environnement/Système est modélisé par un ensemble d'états S
- ❖ Un ensemble d'actions A représente les actions applicables à ces états
- ❖ Apprendre une fonction Q de valeur (quality belief) par état/action : $Q: S \times A \rightarrow \mathbb{R}$
 - ❖ La fonction mesure la qualité d'une action exécutée dans un état donné du système
- ❖ Une politique/stratégie de choix/exploration est appliqué à chaque « épisode » de l'apprentissage:
 - ❖ p.ex. la politique ϵ -greedy

Apprentissage par Renforcement (4)

❖ Stratégie ϵ -greedy

- ❖ Générer un nombre réel aléatoire entre 0 et 1
- ❖ Si ce nombre est inférieur à ϵ , alors choisir une action au hasard
- ❖ Sinon choisir la meilleure action pour l'état courant de l'agent



Apprentissage par Renforcement (5)

- ❖ Algorithme Q-learning (avec stratégie ϵ -greedy)

- ❖ Entrées : taux d'apprentissage $\alpha > 0$, un petit $\epsilon > 0$ et un facteur d'actualisation (discount/cooling factor) $\lambda > 0$

- ❖ Sortie : la fonction $Q: S \times A \rightarrow \mathbb{R}$

- ❖ Initialiser arbitrairement $Q(s_{nt}, a)$ où s_{nt} est un état non terminal pour toute action a

- ❖ Initialiser à zéro $Q(s_t, a)$ où s_t est un état terminal pour toute action a

- ❖ Déroulement d'un épisode d'entraînement:

- ❖ $s \leftarrow$ état initial

- ❖ Répéter

- ❖ Choisir une action a depuis s en utilisant la politique dérivée de Q par la stratégie ϵ -greedy

- ❖ Exécuter l'action a en observant la récompense r et le nouvel état s'

- ❖ Mettre à jour la fonction Q en utilisant l'équation de Bellman:

$$Q_{new}(s, a) = (1 - \alpha)Q_{old}(s, a) + \alpha \left[r + \lambda \max_{a'} Q(s', a') \right]$$

- ❖ $s \leftarrow s'$

- ❖ Jusqu'à ce que s soit un état terminal

Apprentissage par Renforcement (6)

❖ Exemple : Problème du lac gelé

- ❖ L'objectif de l'agent est de partir d'une extrémité (S) d'un lac gelé et d'arriver à une autre extrémité (G) sans tomber dans des trous (H) en empruntant les cases gelées (F)
- ❖ L'agent commence avec une matrice Q initialisée avec des zéros
- ❖ Actions possibles : {aller à gauche (L), aller à droite (R), aller en haut (U), aller en bas (D) }
- ❖ Assignment des récompenses : { $G \rightarrow 100$, $H \rightarrow -100$, $F \rightarrow 0$ }

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

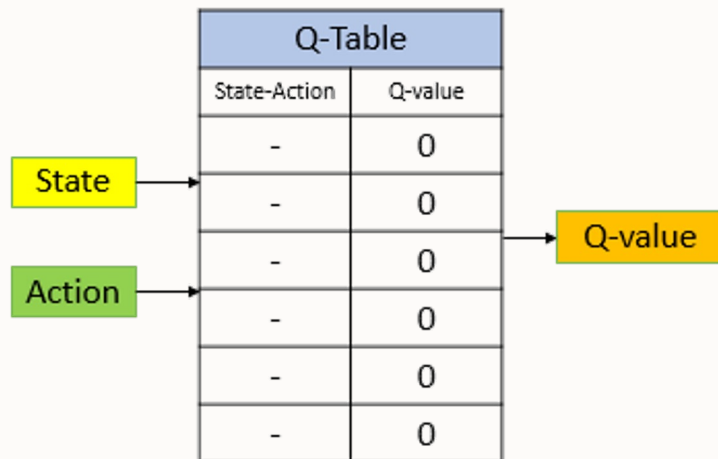
Apprentissage par Renforcement (7)

❖ Algorithme SARSA (avec stratégie ϵ -greedy)

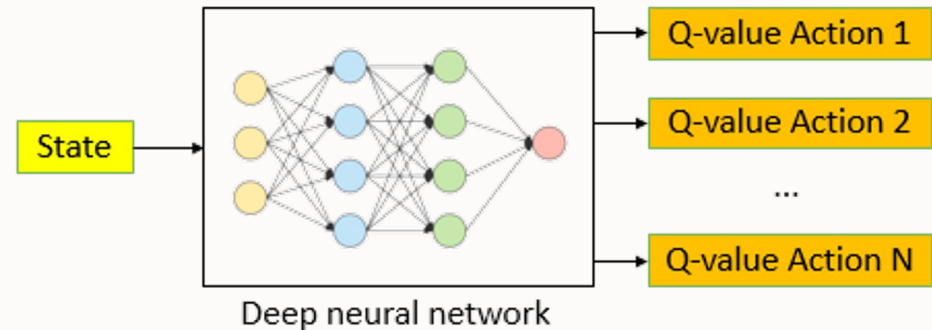
- ❖ Entrées : taux d'apprentissage $\alpha > 0$, un petit $\epsilon > 0$ et un facteur d'actualisation (discount/cooling factor) $\lambda > 0$
- ❖ Sortie : la fonction $Q: S \times A \rightarrow \mathbb{R}$
- ❖ Initialiser arbitrairement $Q(s_{nt}, a)$ où s_{nt} est un état non terminal pour toute action a
- ❖ Initialiser à zéro $Q(s_t, a)$ où s_t est un état terminal pour toute action a
- ❖ Déroulement d'un épisode d'entraînement:
 - ❖ $s \leftarrow$ état initial
 - ❖ Choisir une action a depuis s en utilisant la politique dérivée de Q par la stratégie ϵ -greedy
 - ❖ Répéter
 - ❖ Choisir une action a depuis s en utilisant la politique dérivée de Q par la stratégie ϵ -greedy
 - ❖ Exécuter l'action a en observant la récompense r et le nouvel état s'
 - ❖ Choisir une action a' depuis s' en utilisant la politique dérivée de Q par la stratégie ϵ -greedy:
$$Q_{new}(s, a) = (1 - \alpha)Q_{old}(s, a) + \alpha[r + \lambda \underline{Q(s', a')}]$$
 - ❖ $s \leftarrow s'$
 - ❖ $a \leftarrow a'$
 - ❖ Jusqu'à ce que s soit un état terminal

Apprentissage par Renforcement (8)

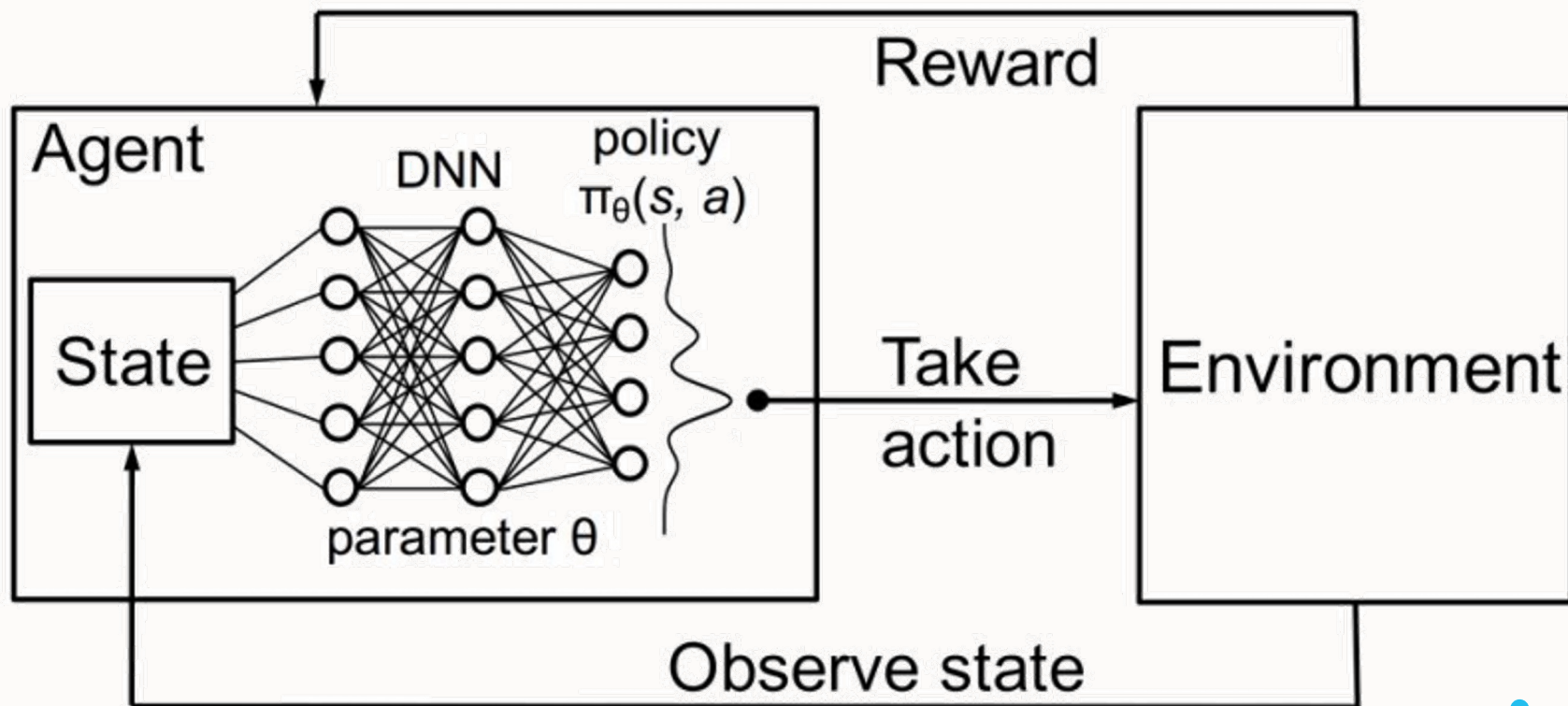
Q-Learning



Deep Q-Learning



Apprentissage par Renforcement (9)



Apprentissage par Renforcement (10)

Start with $Q_0(s, a)$ for all s, a .

Get initial state s

For $k = 1, 2, \dots$ till convergence

Sample action a , get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

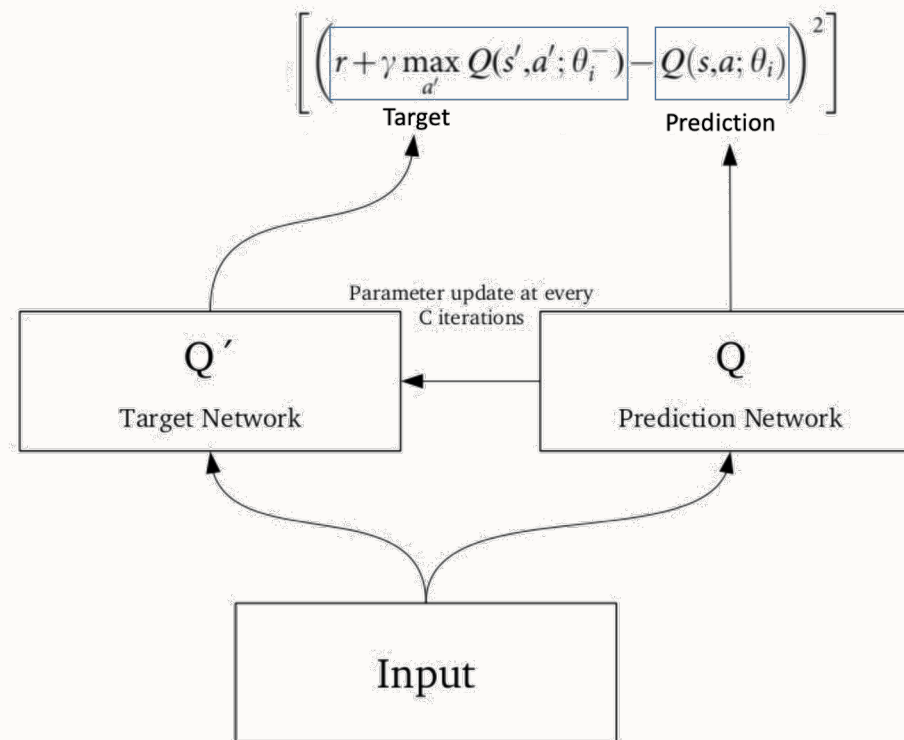
$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathbb{E}_{s' \sim P(s'|s,a)} [(Q_{\theta}(s, a) - \text{target}(s'))^2] \big|_{\theta=\theta_k}$$

$$s \leftarrow s'$$

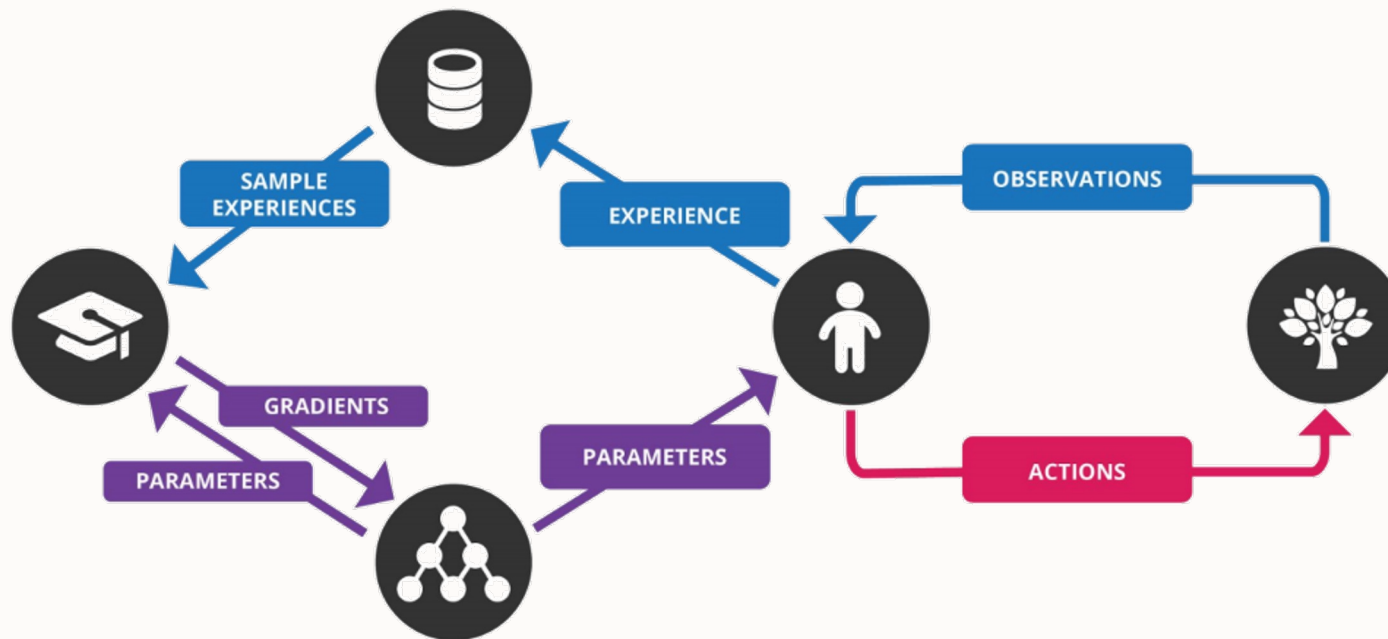
Chasing a nonstationary target!

Updates are correlated within a trajectory!

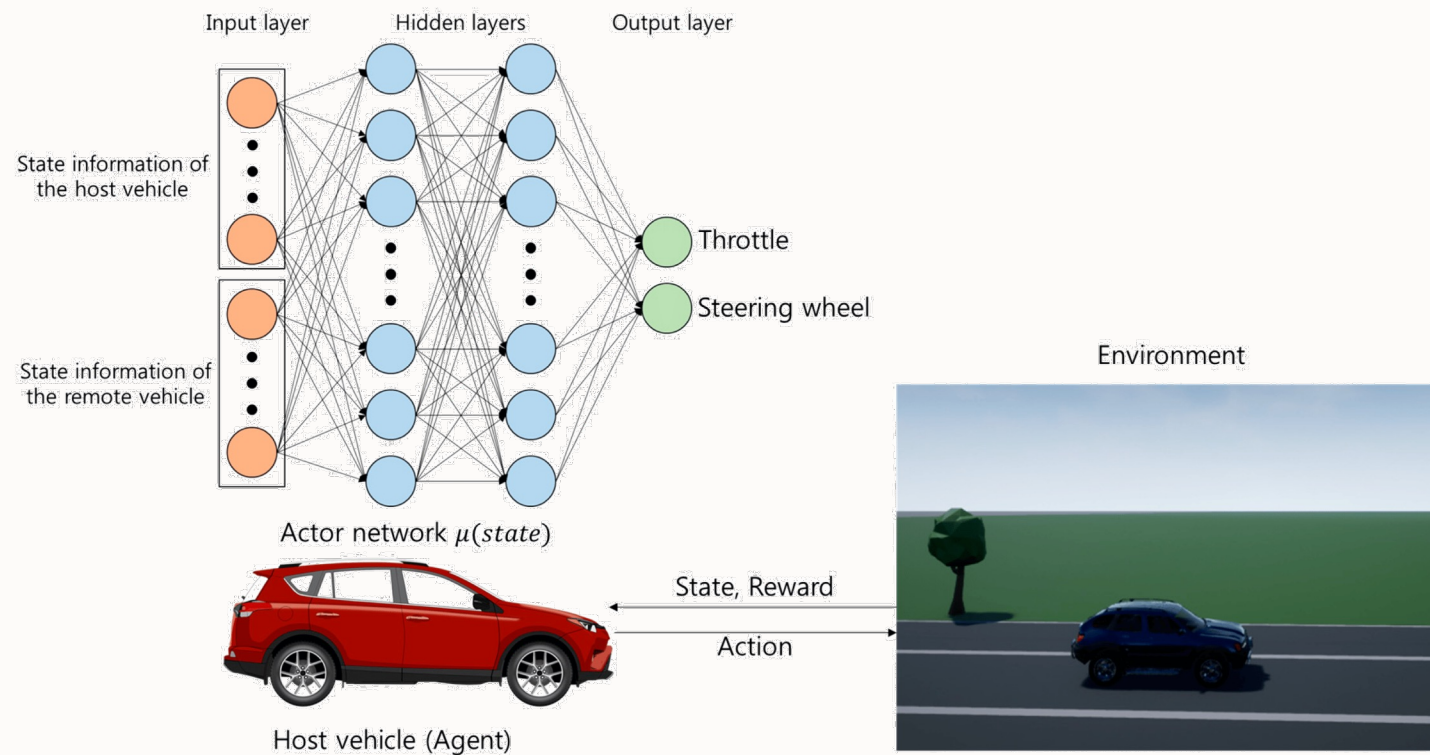
Apprentissage par Renforcement (11)



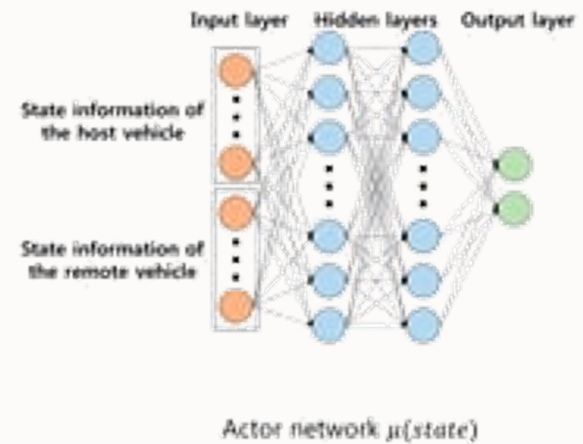
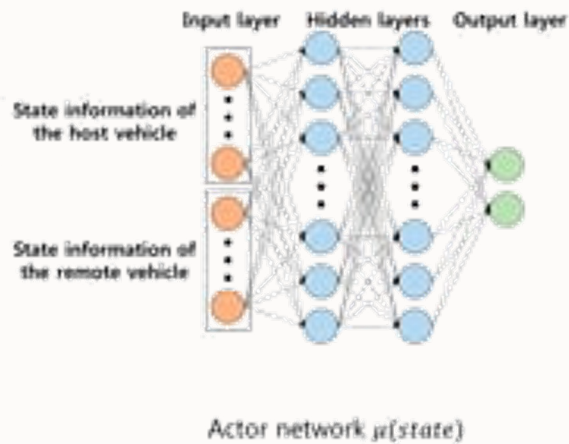
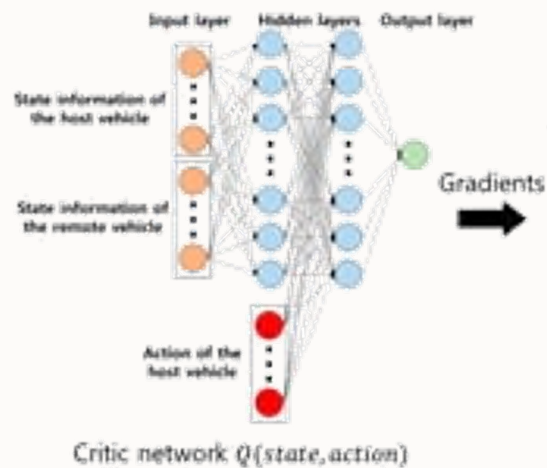
Apprentissage par Renforcement (12)



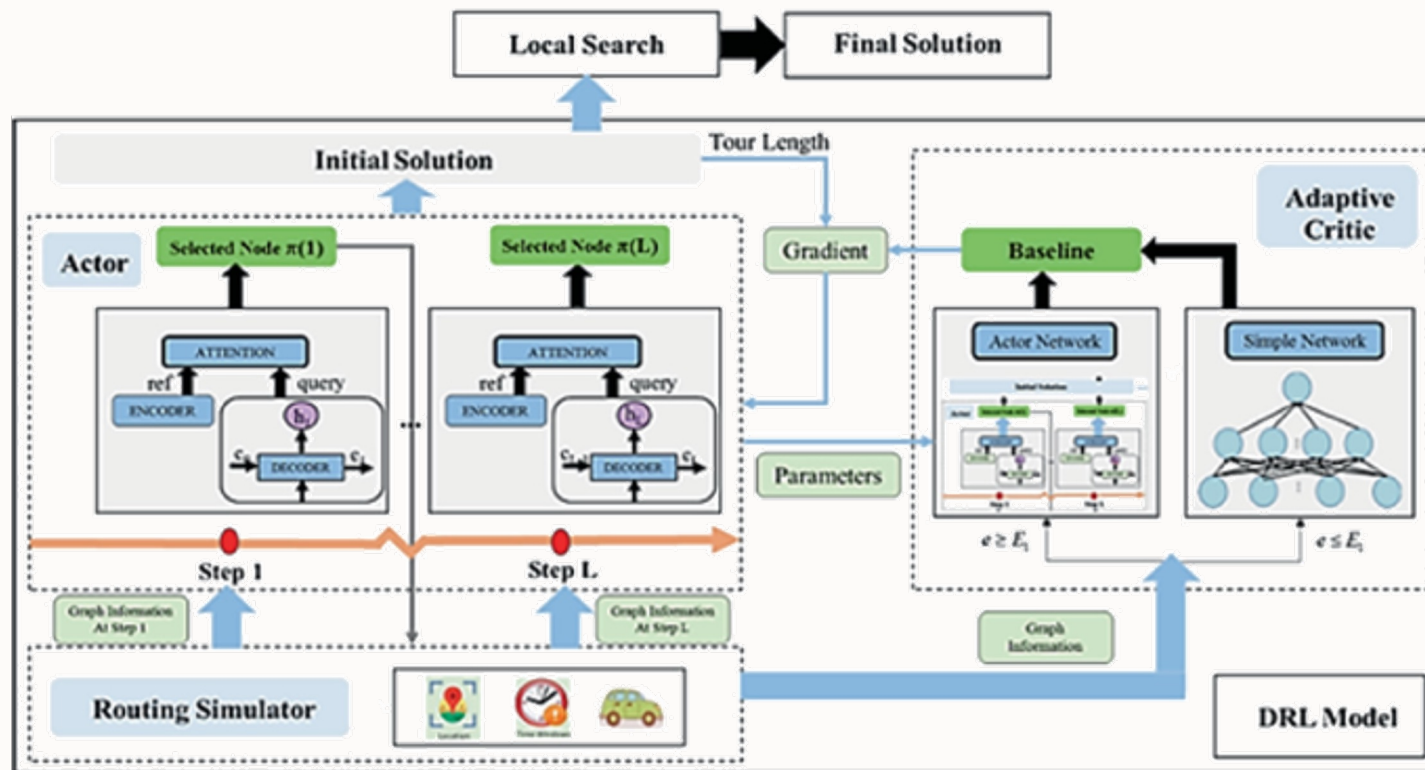
Apprentissage par Renforcement (13)



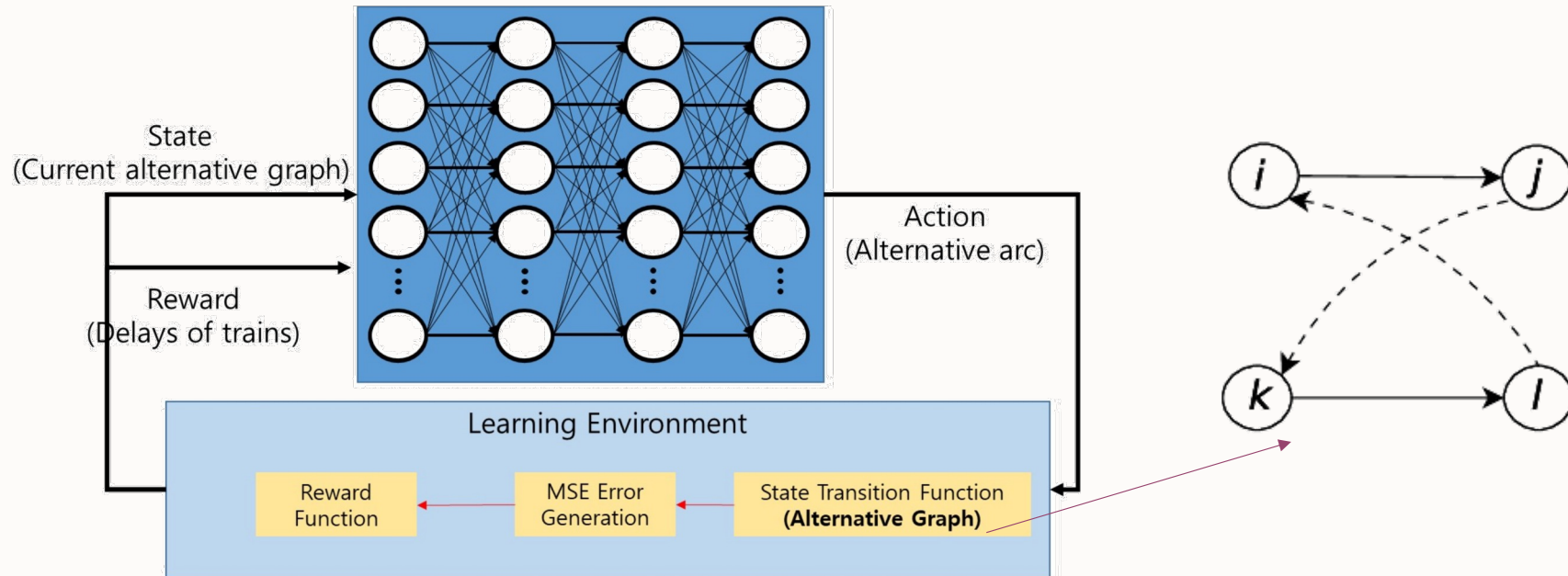
Apprentissage par Renforcement (14)



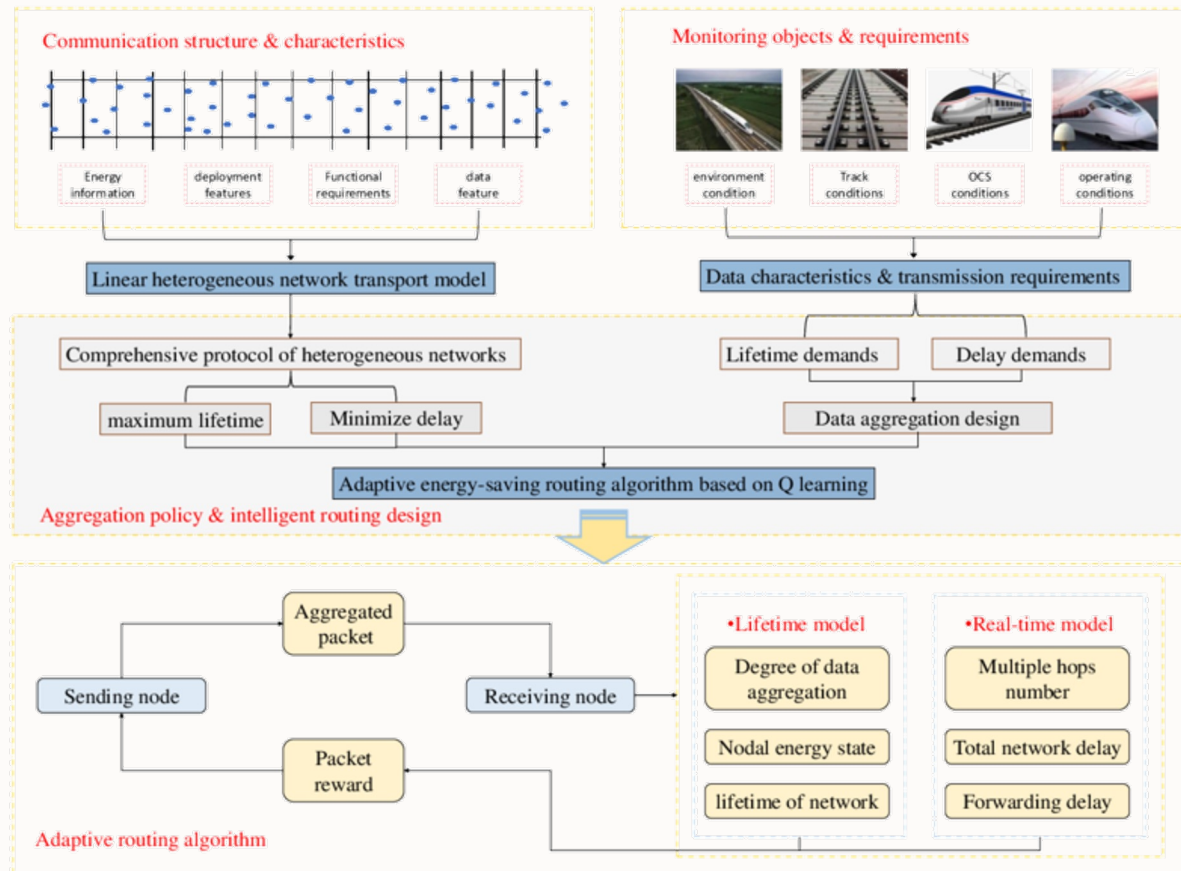
Apprentissage par Renforcement (15)



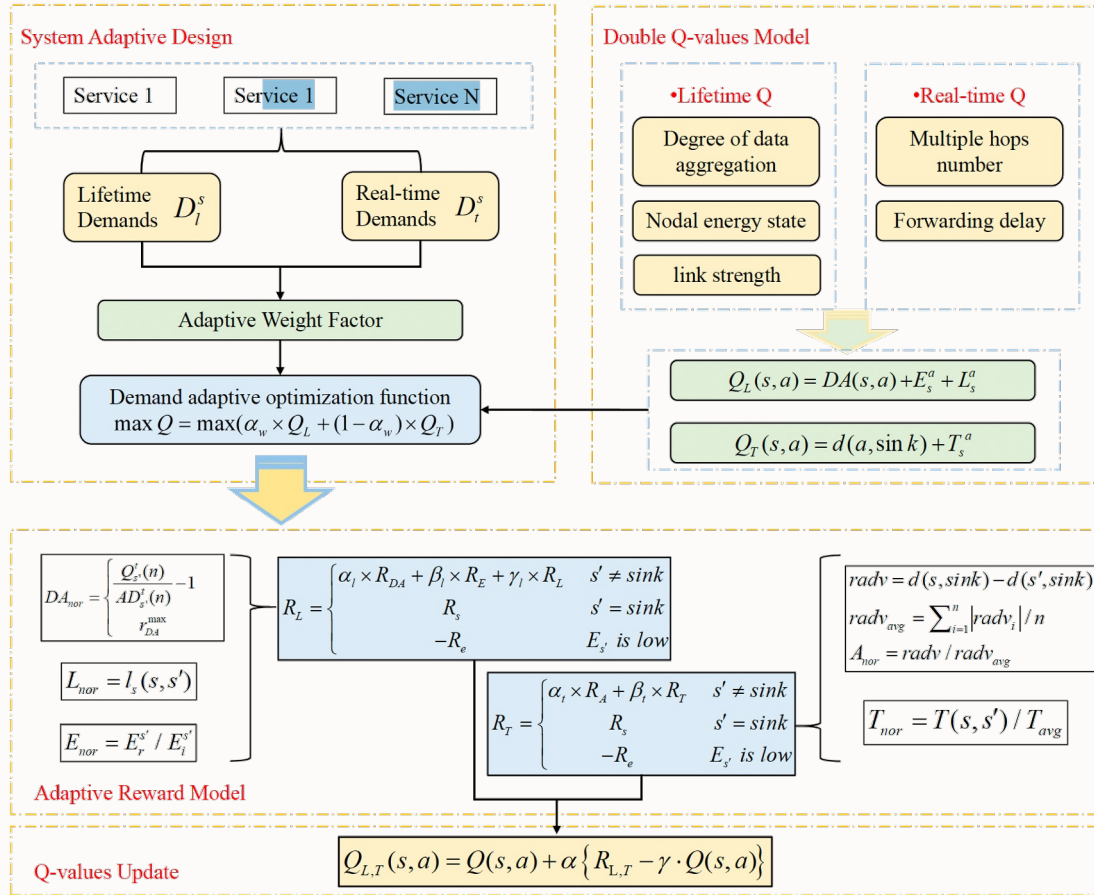
Apprentissage par Renforcement (16)



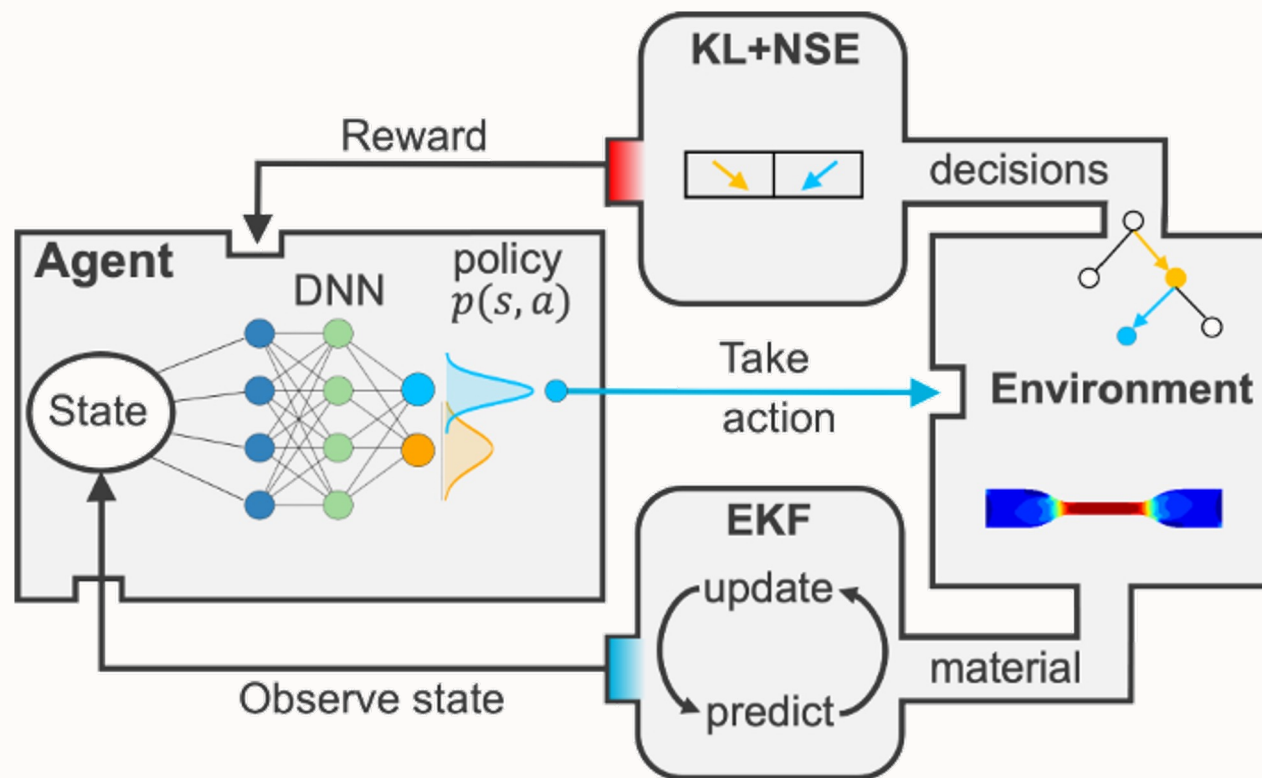
Apprentissage par Renforcement (17)



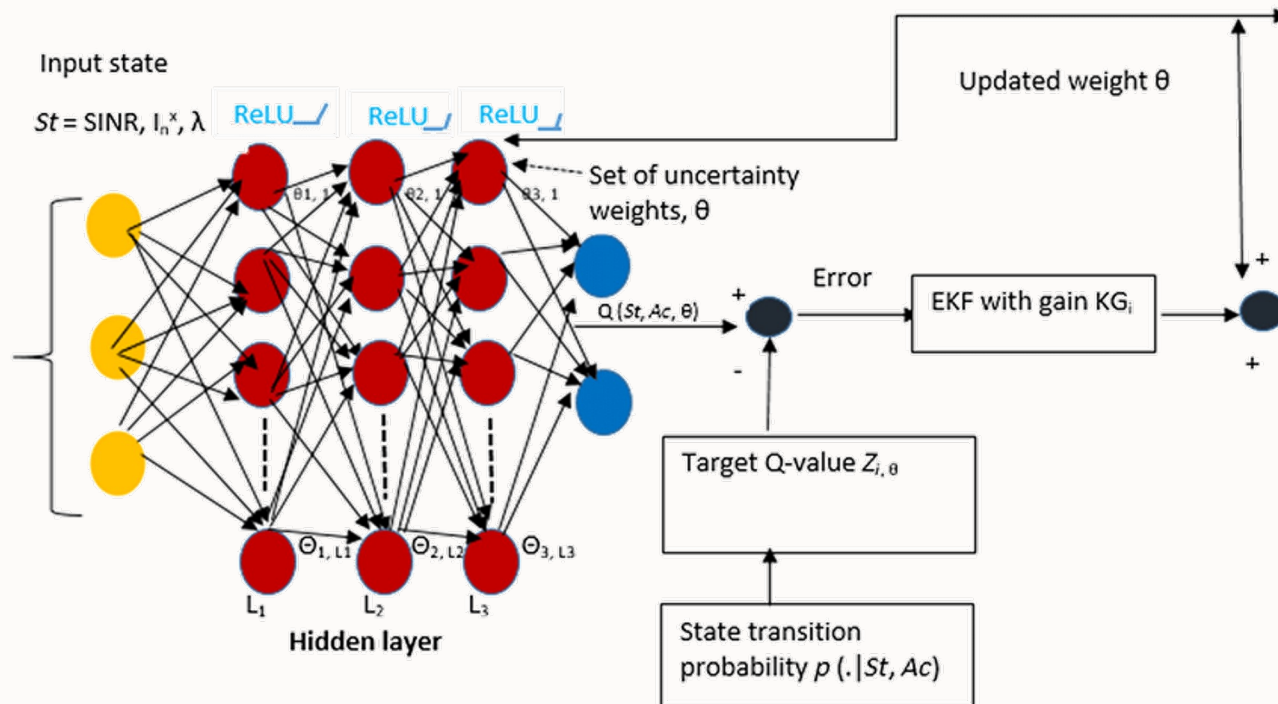
Apprentissage par Renforcement (18)



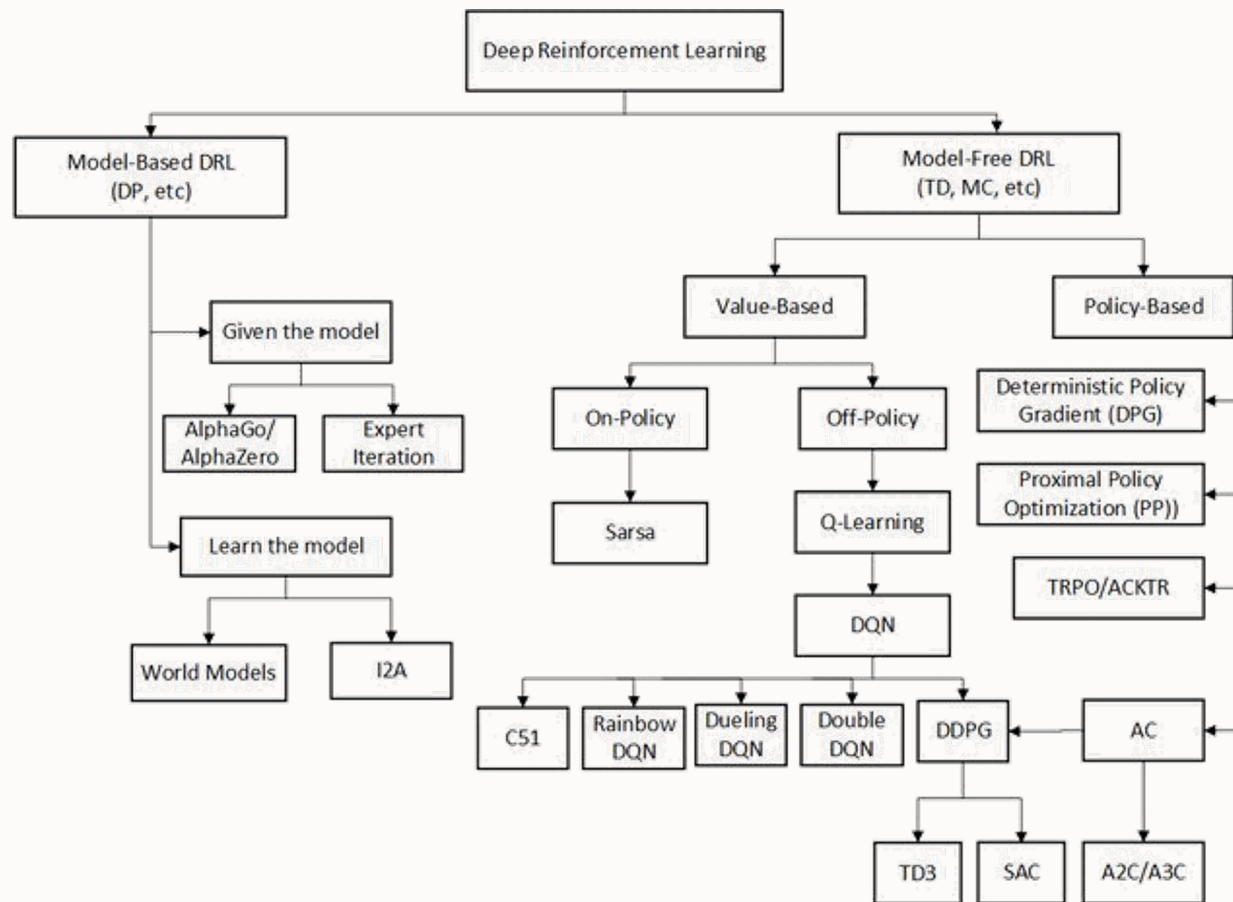
Apprentissage par Renforcement (19)



Apprentissage par Renforcement (20)



Apprentissage par Renforcement (21)



Apprentissage par Renforcement (22)

❖ Résolution du problème du lac gelé avec Python : (Configuration #1)

```
import gymnasium as gym
```

```
import numpy as np
```

```
desc = ["SFFF", "FHFH", "FFFH", "HFFG"]
```

```
MAX_ITERATIONS = 19
```

```
NUM_EPISODES = 10000
```

```
alpha = 0.7
```

```
gamma = 0.97
```

```
epsilon = 1.0
```

```
decay = NUM_EPISODES // 100
```

```
Q = np.zeros((16, 4))
```

```
env = gym.make('FrozenLake-v1', desc=desc, map_name="4x4", is_slippery=False, render_mode="human")
```

Apprentissage par Renforcement (23)

```
def epsilon_greedy(state) -> int:
    p = np.random.random()
    if p < epsilon:
        return env.action_space.sample()
    return np.argmax(Q[state])

def q_learning():
    global epsilon
    for episode in range(NUM_EPISODES):
        if episode != 0 and episode % decay == 0:
            epsilon -= 0.01
        print(f"episode #{episode}: epsilon = {epsilon}, total reward = ", end="")
        state, _ = env.reset()
        env.render()
        total_reward = 0
        visited = set()
        for _ in range(MAX_ITERATIONS):
            visited.add(state)
            action = epsilon_greedy(state)
            new_state, reward, terminated, truncated, info = env.step(action)
            env.render()
            reward = float(reward)
            if new_state in [5, 7, 11, 12] or state == new_state:
                reward = -1.0
            if new_state not in [5, 7, 11, 12] and state != new_state and new_state in visited:
                reward = -0.25
            total_reward += reward
            Q[state][action] = (1 - alpha) * Q[state][action] + alpha * (reward + gamma * np.max(Q[new_state]))
            if terminated or truncated:
                break
            state = new_state
        print(total_reward)
        print(Q)

if __name__ == '__main__':
    q_learning()
```

Apprentissage par Renforcement (24)

episode #740: epsilon = 0.9299999999999999, total reward = -8.25

```

0 [ [-0.29350503  0.67875771  0.73510065 -0.31386078]
1 [  0.44157218 -1.          0.78099626 -0.28610987]
2 [  0.49221508  0.82396377  0.42475762 -0.21267191]
3 [  0.49506098 -1.          -0.48523036 -0.43275045]
4 [ -0.29328665  0.57495108 -1.          0.3931389  ]
5 [  0.          0.          0.          0.          ]
6 [ -1.          0.86626657 -1.          0.5123063  ]
7 [  0.          0.          0.          0.          ]
8 [ -0.28474633 -1.          0.6832523  0.45474762]
9 [  0.57111643  0.81580536  0.89296193 -1.          ]
10 [  0.79453573  0.96527397 -1.          0.59684278]
11 [  0.          0.          0.          0.          ]
12 [  0.          0.          0.          0.          ]
13 [ -1.          -0.13718462  0.90160981  0.56989019]
14 [  0.84012569 -0.03000005  1.          0.74853002]
15 [  0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (25)

episode #1055: epsilon = 0.8999999999999999, total reward = -2.0

```

0 [[-0.22379878  0.82296517  0.58341183 -0.23500155]
1 [ 0.52630313 -1.          0.65558009 -0.36599404]
2 [ 0.43801812  0.78458075  0.38724873 -0.34454296]
3 [ 0.40449877 -1.          -0.58502229 -0.61254894]
4 [-0.47331758  0.84897218 -1.          0.48393047]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.90491322 -1.          0.55882999]
7 [ 0.          0.          0.          0.          ]
8 [-0.16149767 -1.          0.87820737  0.60972936]
9 [ 0.76334279  0.58089459  0.9222833 -1.          ]
10 [ 0.78549196  0.9541224 -1.          0.64405232]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.22460525  0.77923717  0.63723315]
14 [ 0.67137729 -0.03       1.          0.6712299 ]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	E_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (26)

episode #1271: epsilon = 0.8799999999999999, total reward = 1.0

```
0 [[-0.22054473  0.83092341  0.34006002 -0.22908219]
1 [ 0.51766467 -1.          0.64634281 -0.33521668]
2 [ 0.2974418   0.78118007  0.2488826  -0.30185423]
3 [ 0.44662283 -1.          -0.57800437 -0.57943022]
4 [-0.17493489  0.87011378 -1.          0.54389843]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.83243552 -1.          0.44058908]
7 [ 0.          0.          0.          0.          ]
8 [-0.13287564 -1.          0.90856917  0.5489839 ]
9 [ 0.60274717  0.94008041  0.85383251 -1.          ]
10 [ 0.86617862  0.77396873 -1.          0.55756137]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.2756893  0.96983381  0.66196819]
14 [ 0.89476335 -0.03       1.          0.59213168]
15 [ 0.          0.          0.          0.          ]]
```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (27)

episode #2000: epsilon = 0.7999999999999998, total reward = -5.0

```
0 [[-0.27002688  0.77651964  0.47122093 -0.27135804]
1 [ 0.29272995 -1.          0.24442637 -0.17096503]
2 [ 0.20367196  0.42504094  0.0535714  -0.54184055]
3 [ 0.19680878 -1.          -0.55526531 -0.55557381]
4 [-0.24256517  0.83449928 -1.          0.40342051]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.64820281 -1.          0.47070324]
7 [ 0.          0.          0.          0.          ]
8 [-0.14000441 -1.          0.84951887  0.38549567]
9 [ 0.60466887  0.86422312  0.82193304 -1.          ]
10 [ 0.545078    0.95423821 -1.          0.6398883 ]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.19806268  0.91745015  0.51712685]
14 [ 0.76082141 -0.03      1.          0.60628272]
15 [ 0.          0.          0.          0.          ]]
```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (28)

episode #2575: epsilon = 0.7499999999999998, total reward = 0.5

```
0 [[-0.41866946  0.41086756  0.55307929 -0.41848304]
1 [ 0.42861334 -1.          0.404486  -0.30148098]
2 [ 0.34965573  0.66531161  0.23755584 -0.39945235]
3 [ 0.34755456 -1.          -0.62156849 -0.58335778]
4 [-0.29258098  0.7335699  -1.          0.31712241]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.71559501 -1.          0.44241863]
7 [ 0.          0.          0.          0.          ]
8 [-0.23403825 -1.          0.81180587  0.48256545]
9 [ 0.54362811  0.83740955  0.69897933 -1.          ]
10 [ 0.73286906  0.91277394 -1.          0.41472482]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.23616166  0.90118534  0.51223818]
14 [ 0.65598888 -0.03       1.          0.66039148]
15 [ 0.          0.          0.          0.          ]]
```

S ₀	F ₁	F ₂	F ₃
F ₄	H ₅	F ₆	H ₇
F ₈	F ₉	F ₁₀	H ₁₁
H ₁₂	F ₁₃	F ₁₄	G ₁₅

Apprentissage par Renforcement (29)

episode #5020: epsilon = 0.49999999999999956, total reward = 1.0

```

0 [[-0.40545228  0.66205973  0.7235431 -0.45569118]
1 [ 0.43847943 -1.          0.73300716 -0.32216665]
2 [ 0.41616968  0.67346989  0.29817977 -0.40595636]
3 [ 0.4243437   -1.          -0.64883246 -0.66751236]
4 [-0.35807708  0.81969189 -1.          0.33256418]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.92389381 -1.          0.52568133]
7 [ 0.          0.          0.          0.          ]
8 [-0.19199001 -1.          0.89090949  0.50711278]
9 [ 0.54932892  0.81693948  0.63152539 -1.          ]
10 [ 0.75363013  0.96986889 -1.          0.64718026]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.07736699  0.91746132  0.62990084]
14 [ 0.83559291 -0.03       1.          0.68464476]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

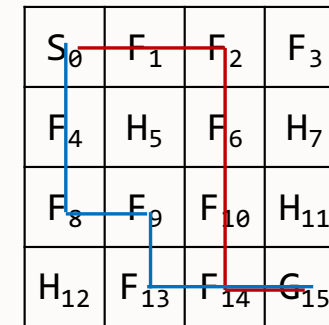
Apprentissage par Renforcement (30)

episode #9999: epsilon = 0.009999999999999247, total reward = 1.0

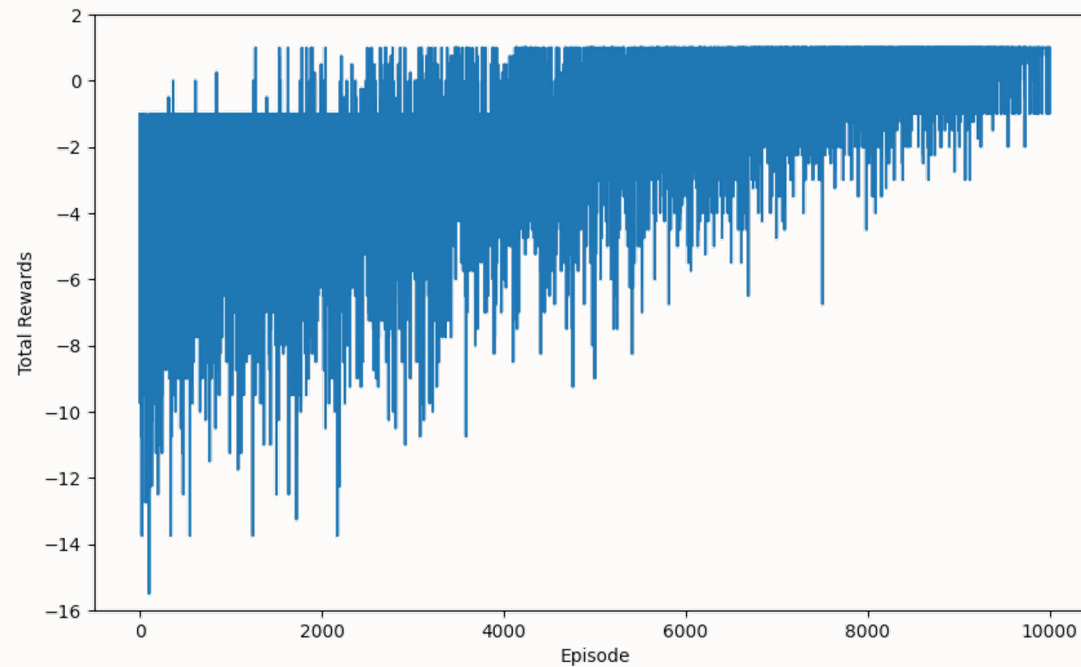
```

0 [[-0.20591446  0.77805185  0.85873403 -0.17156391]
1 [ 0.57216997 -1.          0.88529281 -0.15254293]
2 [ 0.60415168  0.912673    0.61619465 -0.19594021]
3 [ 0.63528886 -1.          -0.40889438 -0.39850487]
4 [-0.1646442   0.81399781 -1.          0.57122156]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.9409     -1.          0.62203557]
7 [ 0.          0.          0.          0.          ]
8 [-0.11573621 -1.          0.896923    0.55979625]
9 [ 0.6362904   0.88839649  0.7659     -1.          ]
10 [ 0.860173    0.97       -1.          0.66219243]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -0.0591191  0.79499991  0.66262877]
14 [ 0.87218102 -0.03       1.          0.75367323]
15 [ 0.          0.          0.          0.          ]]

```



Apprentissage par Renforcement (31)



Apprentissage par Renforcement (32)

❖ Résolution du problème du lac gelé avec Python : (Configuration #2)

```
import gymnasium as gym
import numpy as np

desc = ["SFFF", "FHFH", "FFFH", "HFFG"]
MAX_ITERATIONS = 19
NUM_EPISODES = 10000
alpha = 1.0
gamma = 0.97
epsilon = 0.79
decay = NUM_EPISODES // 100
Q = np.zeros((16, 4))
env = gym.make('FrozenLake-v1', desc=desc, map_name="4x4", is_slippery=False, render_mode="human")

def epsilon_greedy(state) -> int:
    p = np.random.random()
    if p < epsilon:
        action = env.action_space.sample()
        while Q[state][action] == -1.0:
            action = env.action_space.sample()
        return action
    return np.argmax(Q, axis=1)[state]
```

Apprentissage par Renforcement (33)

```
def q_learning():
    global alpha
    for episode in range(NUM_EPISODES):
        if episode != 0 and episode % decay == 0:
            alpha -= 0.01
        print(f"episode #{episode}: alpha = {alpha}, total reward = ", end="")
        state, _ = env.reset()
        env.render()
        total_reward = 0
        visited = set()
        for _ in range(MAX_ITERATIONS):
            visited.add(state)
            action = epsilon_greedy(state)
            new_state, reward, terminated, truncated, info = env.step(action)
            env.render()
            reward = float(reward)
            if new_state in [5, 7, 11, 12] or state == new_state:
                reward = -1.0
            if new_state not in [5, 7, 11, 12] and state != new_state and new_state in visited:
                reward = -0.25
            total_reward += reward
            Q[state][action] = (1 - alpha) * Q[state][action] + alpha * (reward + gamma * np.max(Q[new_state]))
            if terminated or truncated:
                break
            state = new_state
        print(total_reward)
        print(Q)

if __name__ == '__main__':
    q_learning()
```

Apprentissage par Renforcement (34)

episode #17: alpha = 1.0, total reward = -4.25

```

0 [[-1.          -0.235225    0.          -1.          ]
1 [-0.25         -1.          0.          -1.          ]
2 [-0.25         0.60873403  0.          -1.          ]
3 [ 0.582972     -1.          -1.          -1.          ]
4 [-1.           0.          -1.          -0.727725   ]
5 [ 0.           0.           0.           0.          ]
6 [-1.           0.6909      -1.           0.340472   ]
7 [ 0.           0.           0.           0.          ]
8 [-1.           -1.          0.662673    -0.72066825 ]
9 [ 0.64279281   0.9409      0.          -1.          ]
10 [ 0.912673     0.72        -1.          0.60873403 ]
11 [ 0.           0.           0.           0.          ]
12 [ 0.           0.           0.           0.          ]
13 [-1.           -1.          0.97         0.912673   ]
14 [ 0.9409       -1.          1.          0.6909      ]
15 [ 0.           0.           0.           0.          ]]

```

S ₀	F ₁	F ₂	F ₃
F ₄	H ₅	F ₆	H ₇
F ₈	F ₉	F ₁₀	H ₁₁
H ₁₂	F ₁₃	F ₁₄	G ₁₅

Apprentissage par Renforcement (35)

episode #273: alpha = 0.98, total reward = 1.0

```

0 [[-1.          0.16672215  0.84913865 -1.          ]
1 [ 0.55981194 -1.          0.65918705 -1.          ]
2 [ 0.58195517  0.59201772  0.36176237 -1.          ]
3 [ 0.63059192 -1.          -1.          -1.          ]
4 [-1.          0.85742       -1.          0.14217473]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.92658896 -1.          0.60858121]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.68172464  0.15452093]
9 [ 0.4097132   0.45358626  0.45822329 -1.          ]
10 [ 0.43083758  0.969902    -1.          0.67687196]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.72000196  0.43204825]
14 [ 0.44868148 -1.          1.          0.68624152]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (36)

episode #396: alpha = 0.97, total reward = 0.75

```

0 [[-1.          0.63429408  0.38848999 -1.          ]
1 [ 0.15168575 -1.          0.41439374 -1.          ]
2 [ 0.1463501   0.662898    0.15939986 -1.          ]
3 [ 0.39301943 -1.          -1.          -1.          ]
4 [-1.          0.6550923   -1.          0.3579031   ]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.9334     -1.          0.64254718 ]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.89873325  0.14255868 ]
9 [ 0.17830199  0.69068831  0.94068155 -1.          ]
10 [ 0.66267263  0.97       -1.          0.42017978 ]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.72749325  0.64875236 ]
14 [ 0.93318015 -1.          1.          0.93319448 ]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (37)

episode #1163: alpha = 0.8899999999999999, total reward = 1.0

```

0 [[-1.          0.33211411 -0.05278026 -1.          ]
1 [-0.04200627 -1.          0.57658057 -1.          ]
2 [ 0.14455425  0.21514363  0.1560791  -1.          ]
3 [ 0.00475017 -1.          -1.          -1.          ]
4 [-1.          0.47024526 -1.          -0.0133345  ]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.86606585 -1.          0.00921551 ]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.80659196  0.13010498 ]
9 [ 0.46759665  0.87088038  0.8728821  -1.          ]
10 [ 0.81998657  0.96697859 -1.          0.45205425 ]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.74719983  0.440083  ]
14 [ 0.89174645 -1.          1.          0.47249742 ]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	E_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (38)

episode #3011: alpha = 0.6999999999999997, total reward = 1.0

```

0 [[-1.          0.61483732  0.31639412 -1.          ]
1 [ 0.21685896 -1.          0.5543776  -1.          ]
2 [ 0.22799506  0.57513823  0.08022718 -1.          ]
3 [ 0.38865191 -1.          -1.          -1.          ]
4 [-1.          0.72130669 -1.          0.24549699]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.87425874 -1.          0.2962968  ]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.81126615  0.36171697]
9 [ 0.38763418  0.8502572  0.91148144 -1.          ]
10 [ 0.54150381  0.96857188 -1.          0.59134283]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.94754264  0.53725361]
14 [ 0.51188515 -1.          1.          0.83981567]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

Apprentissage par Renforcement (39)

episode #5036: alpha = 0.49999999999999956, total reward = 0.25

```

0 [[-1.          0.33960253  0.36066092 -1.          ]
1 [ 0.25851329 -1.          0.4404319 -1.          ]
2 [ 0.24826046  0.37870169  0.21361564 -1.          ]
3 [ 0.32813381 -1.          -1.          -1.          ]
4 [-1.          0.36309172 -1.          0.24347763]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.68171195 -1.          0.32397346]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.56813259  0.28917418]
9 [ 0.43153157  0.72408733  0.69460677 -1.          ]
10 [ 0.68748373  0.91183337 -1.          0.33990793]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.9466538  0.53969013]
14 [ 0.60548624 -1.          1.          0.52591221]
15 [ 0.          0.          0.          0.          ]]

```

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

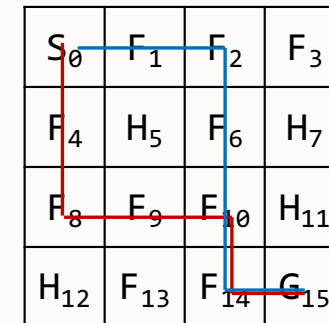
Apprentissage par Renforcement (40)

episode #9999: alpha = 0.0099999999999999247, total reward = 0.0

```

0 [[-1.          0.40392767  0.39203658 -1.          ]
1 [ 0.13411672 -1.          0.49894121 -1.          ]
2 [ 0.25629936  0.61982071  0.24461503 -1.          ]
3 [ 0.3545012   -1.          -1.          -1.          ]
4 [-1.          0.52861277 -1.          0.13761024 ]
5 [ 0.          0.          0.          0.          ]
6 [-1.          0.7378363   -1.          0.41215059 ]
7 [ 0.          0.          0.          0.          ]
8 [-1.          -1.          0.64547663  0.27128264 ]
9 [ 0.41430727  0.74422801  0.7701065  -1.          ]
10 [ 0.57401303  0.90152764 -1.          0.58386424 ]
11 [ 0.          0.          0.          0.          ]
12 [ 0.          0.          0.          0.          ]
13 [-1.          -1.          0.83312063  0.50728484 ]
14 [ 0.6946654   -1.          1.          0.63767672 ]
15 [ 0.          0.          0.          0.          ]]

```



Apprentissage par Renforcement (41)

